Virtualization → Containers

Timeline, Development, Adoption, and Licensing

1. The Original Problem: Hardware Utilization (1990s)

Before virtualization

- One OS per physical machine
- Low utilization (5–15%)
- Hard to test multiple environments
- High hardware cost

This drove **hardware virtualization**.

2. Virtualization Era (1999–2008)

2.1 VMware – Commercial Virtualization Pioneer

Aspect	Details
First release	**1999**
Type	Type-2 (Workstation), later Type-1 (ESX)
License	**Proprietary / Closed source**
Cost	Paid (trial versions existed)

Why VMware mattered

- First mainstream x86 virtualization
- Enterprise-grade isolation
- Data-center adoption

Limitations

- Heavy (full OS per VM)
- Licensing cost
- Slower provisioning

2.2 QEMU – Open Source Emulation & Virtualization

Aspect	Details
First release	**2003**
License	**GPL / LGPL**
Type	Emulator + virtualizer
Open source	Yes

Key characteristics

- Can **emulate different CPU architectures**
- With **KVM**, becomes near-native virtualization
- Foundation of many platforms

Adoption

- Linux power users
- Cloud providers
- Embedded & cross-architecture testing

2.3 VirtualBox → Oracle VirtualBox

Aspect	VirtualBox (Sun)	Oracle VirtualBox
First release	**2007**	**2010 (rename)**
License	Mixed	Mixed
Core engine	GPLv2	GPLv2
Extension pack	Proprietary	Proprietary

Important

- VirtualBox and Oracle VirtualBox are the **same product**
- Extension Pack adds USB 2/3, RDP, PXE (closed source)

Adoption

- Developers
- Education
- Desktop virtualization

3. Virtualization Maturity → Container Pressure (2008)

Problems VMs could not solve

- Slow boot (minutes)
- High RAM/CPU overhead
- Poor microservice scalability

This created demand for **lightweight isolation**.

4. Containers Begin: LXC (2008)

Aspect	Details
First release	**2008**
License	**GPL / LGPL**
Type	OS-level virtualization
Open source	Yes

Why LXC mattered

- First real Linux containers
- Near-native performance
- No full OS overhead

Why it didn’t explode

- Poor UX
- No image standard
- Sysadmin-focused

5. Docker – Container Adoption Breakthrough (2013)

Aspect	Details
First release	**2013**
Core engine	Open source
Docker Desktop	Proprietary
Business model	Open core

What Docker added

- Dockerfile
- Image layers
- Registry
- Simple CLI

Docker made containers **usable by developers**, not just sysadmins.

6. Standardization & Orchestration (2015–2016)

OCI (Open Container Initiative)

- Prevented Docker lock-in
- Defined runtime & image standards

Kubernetes

- Container orchestration at scale
- Runtime-agnostic

7. Proxmox – Bridging VMs and Containers (2008)

Aspect	Details
First release	**2008**
License	**AGPLv3**
Type	Virtualization platform
Technologies	KVM + LXC
Open source	Yes

Why Proxmox matters

- Runs **VMs and containers together**
- Web UI
- Cluster support

- Popular VMware alternative

Adoption

- SMBs
- Homelabs
- Cost-sensitive enterprises

8. Podman – Enterprise-Grade Containers (2018)

Aspect	Details
First release	**2018**
License	**Apache 2.0**
Open source	Yes
Maintainer	Red Hat

Why Podman exists

- No daemon
- Rootless containers
- OCI compliant

9. Combined Timeline (Virtualization + Containers)

```
1999  → VMware
2003  → QEMU
2007  → VirtualBox
2008  → LXC
2008  → Proxmox
2013  → Docker
2015  → OCI
2018  → Podman
```

10. Licensing Matrix (Clear & Practical)

Technology	Type	License	Open Source	Free to Use	Closed Parts
VMware	VM	Proprietary	No	Limited	Yes
QEMU	VM/Emu	GPL/LGPL	Yes	Yes	No
VirtualBox	VM	GPLv2	Partial	Yes	Extension Pack
Oracle VirtualBox	VM	GPLv2	Partial	Yes	Extension Pack
Proxmox	VM + CT	AGPLv3	Yes	Yes	No
LXC	Container	GPL/LGPL	Yes	Yes	No
Docker Engine	Container	Open source	Yes	Yes	No
Docker Desktop	Tooling	Proprietary	No	Limited	Yes

Podman	Container	Apache 2.0	Yes	Yes	No

11. Why Containers Did NOT Replace VMs

Use Case	Best Choice
Different kernels	VM
Legacy OS	VM
Strong isolation	VM
Microservices	Container
Fast CI/CD	Container
Mixed workloads	Proxmox

12. Final Mental Model

- **VMware** → Enterprise VM pioneer
- **QEMU/KVM** → Open foundation
- **VirtualBox** → Desktop convenience
- **Proxmox** → VM + Container convergence
- **LXC** → Container foundation
- **Docker** → Mass adoption
- **Podman** → Secure, enterprise future

Containers: Timeline, Development, Adoption, and Licensing

1. Before Containers: The Problem They Solved

Traditional deployment (pre-2008)

- Applications ran:
    - Directly on bare metal, or
    - Inside **virtual machines**
- Problems:
    - Heavy resource usage (each VM = full OS)
    - Slow startup
    - Environment mismatch (“works on my machine”)

This led to the need for **lightweight isolation**.

2. Foundations: Linux Isolation Technologies (2000–2008)

Containers did **not** appear suddenly.

Key kernel features

Feature	Introduced	Purpose
`chroot`	1979	Filesystem isolation
Namespaces	2002–2008	Process, network, user isolation
cgroups	2006	Resource limits (CPU, RAM, IO)

These primitives enabled containers.

3. LXC – The First Real Containers (2008)

What is LXC?

- **LXC (Linux Containers)** is the **first practical container system**
- Directly exposes kernel features
- No image format, no registry, no developer UX

Timeline & license

Item	Details
First release	**2008**
License	**LGPL / GPL**
Type	Fully open source
Target users	System administrators

Why LXC didn’t go mainstream

- Complex CLI
- No standardized image workflow
- Hard to distribute applications

LXC was **powerful but unfriendly**.

4. Docker – Container Adoption Explosion (2013)

What Docker changed (not invented)

Docker **did not invent containers**. It **productized** them.

Key innovations

- Dockerfile (reproducible builds)
- Layered images
- Image registry (Docker Hub)
- Simple CLI
- Portable application packaging

Timeline & license

Item	Details
First release	**2013**

Core engine	Open source (Apache 2.0 → later mixed, now Moby)
Docker Desktop	**Proprietary**
Business model	Open core + closed tooling

Adoption phase

- 2014–2017: Massive adoption
- Became default DevOps standard
- Cloud-native ecosystem formed around Docker

5. Container Standardization (2015–2016)

Docker’s dominance created a risk: **vendor lock-in**

OCI (Open Container Initiative)

- Founded by Docker, Red Hat, CoreOS, others
- Defined:
    - Image format
    - Runtime spec

This allowed multiple runtimes to coexist.

6. Podman – Secure, Daemonless Containers (2018)

Why Podman was created

- Docker daemon runs as root
- Security concerns in enterprises
- Red Hat wanted a **drop-in Docker alternative**

Key differences

- No daemon
- Rootless containers
- Uses OCI standards
- CLI compatible with Docker

Timeline & license

Item	Details
First release	**2018**
License	**Apache License 2.0**
Type	Fully open source
Maintainer	Red Hat

Adoption

- Enterprise Linux (RHEL, Fedora)
- Kubernetes-friendly
- Security-sensitive environments

7. High-Level Timeline (Chronological)

```
1979  → chroot
2002  → Linux namespaces
2006  → cgroups
2008  → LXC
2013  → Docker
2015  → OCI standard
2018  → Podman
```

8. Licensing Comparison (Clear and Practical)

Container Technologies

Technology	License	Open Source	Free to Use	Closed Parts
LXC	LGPL / GPL	Yes	Yes	No
Docker Engine	Open source	Yes	Yes	No
Docker Desktop	Proprietary	No	Free (limited)	Yes
Podman	Apache 2.0	Yes	Yes	No

Meaning of terms

- **Open source**: Source code available, modifiable
- **Free to use**: No cost, may still be closed source

- **Closed source**: Source code not available

9. Containers vs Virtual Machines (Why Containers Won)

Aspect	VM	Container
OS	Full OS per VM	Shared kernel
Startup time	Minutes	Seconds
Resource usage	High	Low
Portability	Medium	Very high
Cloud-native	Poor	Excellent

10. Current State (Today)

- **Docker**: Developer UX leader
- **Podman**: Secure, enterprise-focused
- **LXC**: Still used, mostly low-level/system use
- **Kubernetes**: Orchestrates containers, runtime-agnostic

Containers are now **infrastructure primitives**, not tools.

11. Summary

Containers evolved from Linux kernel isolation (LXC, 2008), were popularized by Docker (2013) through developer-friendly tooling, standardized via OCI, and matured with secure alternatives like Podman (2018). All major container runtimes are **open source**, but some surrounding tools (notably Docker Desktop) are **proprietary**. Containers won because they maximize portability, speed, and resource efficiency.